



INSTITUTO DE
INFORMÁTICA
UFG

Compiladores

Thierson
Couto Rosa

Compiladores

Thierson Couto Rosa

Instituto de Informática
Universidade Federal de Goiás

19 de fevereiro de 2026



O que é um Compilador?

- Programa que lê um programa escrito em uma linguagem de programação (linguagem fonte) e o traduz para outra linguagem (linguagem objeto) de tal modo que o programa obtido é funcionalmente semelhante ao programa original em linguagem fonte.
- Durante a tradução o compilador relata a presença de erros.
- O compilador não executa o programa fonte.



Aplicabilidade das técnicas estudadas em Compiladores

- Implementação de tradutores:
 - Shell, interpretadores embutidos em softwares (Maple, Project R, etc).
 - Comentários javadoc para html.
 - Gerar tabelas a partir de consultas SQL.
 - Interpretadores de protocolos de comunicação (http, ftp, etc).
 - Interpretadores para impressoras (Postscript).
 - Compiladores para HDL (Hardware Description Language).
 - Sistemas de informação com regras que precisam ser reprogramadas periodicamente (filtros ante-spam - SpamAssassin, folha de pagamento!).



Modelo de Compilação - Como um compilador trabalha?

- O compilador trabalha em duas etapas:
 - Etapa de análise (front-end)
 - Divide o programa fonte em suas partes constituintes.
 - Verifica se o programa fonte está de acordo com as regras léxicas, sintáticas e semânticas.
 - Gera uma representação intermediária do programa fonte.
 - Etapa de síntese (back-end)
 - Otimização da representação intermediária.
 - Tradução da representação intermediária para a linguagem objeto.
 - Otimização do código em linguagem objeto.



Introdução

Compiladores

Thierson
Couto Rosa

Etapas

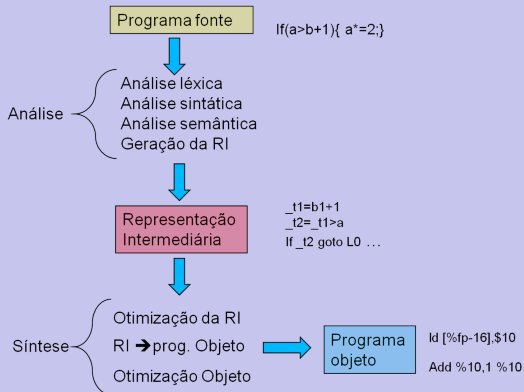


Figura: Ilustração das etapas



Visão Geral

- Percorre o texto do programa fonte obtendo átomos (tokens ou símbolos terminais) da linguagem fonte
- Ex: `int a; a=a+2;`
- Análise léxica (*scanning*):
 - `int` T_int
 - `a` T_identificador
 - `;` T_pv.



Visão Geral

- Os símbolos terminais tokens são “encaixados” nas produções de uma gramática LC da linguagem fonte.
 - $\langle Decl \rangle \rightarrow \langle Tipo \rangle \langle ListaIdentificador \rangle T_{pv}$
 - $\langle Tipo \rangle \rightarrow T_{int} \mid T_{float} \mid T_{char}$
 - $\langle Expr \rangle \rightarrow \langle Expr \rangle T_{opMais} \langle Expr \rangle$
- Gera no final uma árvore sintática abstrata.
- Erros sintáticos: a sequência de tokens não se “encaixa” nas produções da GLC.



Visão Geral

- Percorre a árvore sintática gerada verificando a existência de erros semânticos:
 - Identificador não declarado
 - Tipos incompatíveis



Geração da representação intermediária

- A representação deve ser simples para facilitar a geração do código objeto que na maioria das vezes é uma linguagem de baixo nível (assembly ou linguagem de máquina).
- Existem diversas representações intermediárias (máquina de Pillha, Three Address Code (TAC), árvore sintática abstrata)



Exemplos de representação Intermediária

Programa Fonte

```
a=b*c+b*d
```



Representação Intermediária

```
_t1=b*c  
_t2=b*d  
_t3=_t1+_t2  
a=_t3
```

Programa Fonte

```
if(a<=b) a=a-c; c=b*c;
```



Representação Intermediária

```
_t1=a>b  
If(_t1) goto L0  
_t2=a-c  
a=_t2  
L0: _t3=b*c  
c=_t3
```

Figura: Exemplos de representação intermediária



Otimizações

- Inibir geração de trecho de código inatingível.
- Eliminar variáveis que não são utilizadas.
- Eliminar de operadores neutros ($x*1$; $x+0$).
- Otimizar comandos de repetição.



Exemplo de otimização da RI

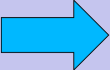
<code>_t1=b*c</code>		<code>_t1=b*c</code>
<code>_t2=_t1+0</code>		<code>_t2=_t1+_t1</code>
<code>_t3=b*c</code>		<code>a=_t2</code>
<code>_t4=_t2+_t3</code>		
<code>a=_t4</code>		

Figura: Exemplo de otimização da RI

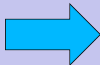


Geração de Código

Traduzir de RI para a linguagem objeto

Representação Intermediária

```
_t1=b*c  
_t2=_t1+_t1  
a=_t2
```



Linguagem Objeto

```
ld[%fp-16],%l1  
ld[%fp-20],%l2  
smul %l1,%l2,%l3  
add %l3,%l3,%l0  
st %l0 [%fp-24]
```

Figura: Tradução de RI para a linguagem objeto



Otimização do código objeto

- Assim como a geração do código é altamente dependente de arquitetura.
- Otimização do uso de registradores.
- Aproveitar de recursos da arquitetura como pipeline.
- Previsão de desvio.



Conhecimentos Teóricos

- Análise léxica:
 - Revisão de Expressões Regulares e Autômatos Finitos.
 - Como utilizar teoremas de LFA para construir geradores de analisadores léxicos.
 - Entender como um gerador de analisador léxico (ex. *Flex*, *Lex*) pode ser construído.
- Análise Sintática:
 - Revisão de gramáticas livres de Contexto - GLC
 - Estudo de GLC apropriadas para linguagens de programação: LL(k), SLR, LR, LALR.
 - Entender como um gerador de analisador sintático (Bison, Yacc, etc) pode ser construído.
- Conhecimentos vindos de outras disciplinas: Estruturas de Dados.



Visão Geral da Disciplina

Compiladores

Thierson
Couto Rosa

A disciplina está dividida em duas partes.

- Parte 1 - As aulas desta parte são direcionadas para dar aos alunos condições de implementar um compilador para uma linguagem simples a G-V1
 - Uso de ferramentas geradoras de analisadores léxicos e sintáticos sem se preocupar em como essas ferramentas são implementadas.
 - Teoria necessária para gerar tradução dirigida por sintaxe do programa fonte em uma árvore sintática abstrata.
 - Explicação sobre tabelas de símbolos.
 - Explicação sobre geração de código suficiente para a linguagem G-V1.

Objetivo: propiciar uma visão completa das principais fases de um compilador ainda no meio da disciplina. Esta etapa corresponde ao trabalho prático 1.



Visão Geral da Disciplina

Compiladores

Thierson
Couto Rosa

- Parte 2 - As aulas desta parte visam aprofundar os conhecimentos de análise léxica, análise sintática, análise semântica e geração de código.
- Serão estudados:
 - Subconjuntos de linguagens livres de contexto apropriadas para a escrita de linguagens de programação.
 - Como implementar ferramentas que geram analisadores léxicos (Flex, Lex e regex).
 - Como implementar ferramentas que geram analisadores sintáticos (Bison, Antlr, Yacc).

Objetivo: propiciar uma visão de como as ferramentas usadas no curso podem ser implementadas. Nesta etapa, os alunos devem implementar um compilador para a linguagem G-V2 que é uma extensão da linguagem G-V1 que inclui: chamada de funções e o tipos de dados array.



Conhecimentos Práticos

- Aprender a utilizar um gerador de analisador léxico (gerador de autômatos) (Fex, Lex) para gerar programas em C/C++ que reconhecem os tokens de uma linguagem de programação.
- Aprender a utilizar um gerador de analisador sintático (Bison, Yacc) para gerar um autômato *pushdown* para uma gramática de uma linguagem de alto nível.
- Gerar os analisadores léxicos, sintáticos, e semânticos para uma linguagem de programação didática.
- Gerar código para uma linguagem de programação didática.